

Numpy and Machine Learning Fundamentals

ZeroToML



Numpy



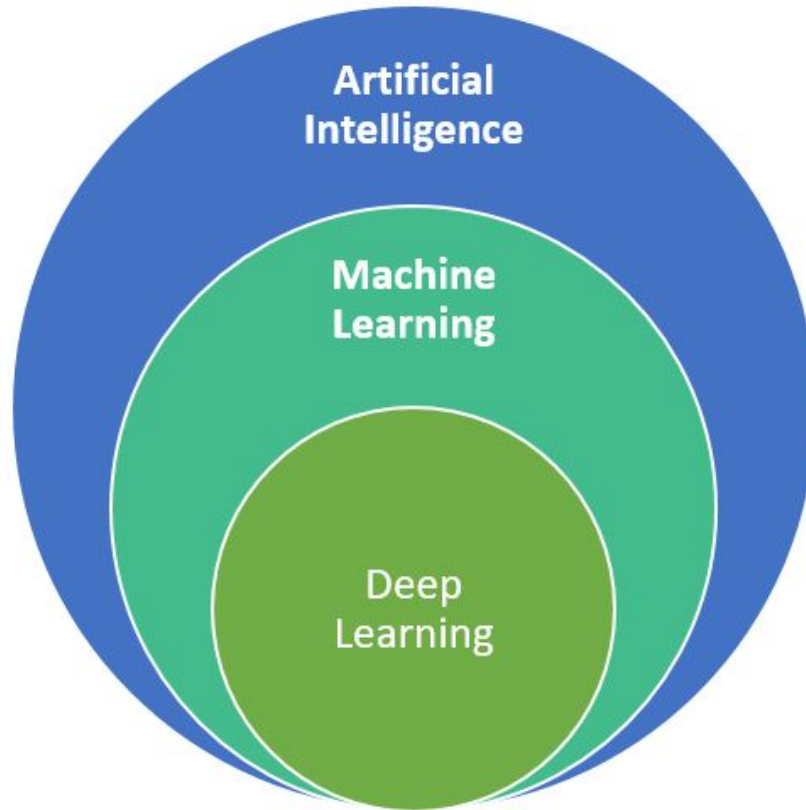
Numpy

pull and create environment



Machine Learning Fundamentals





1. Learns on it's own
2. from data

Figure 1: artificial intelligence, machine leaning and deep learning Source: Nadia BERCHANE (M2 IESCI, 2018)



Supervised ML



Features:
input



ML
Model

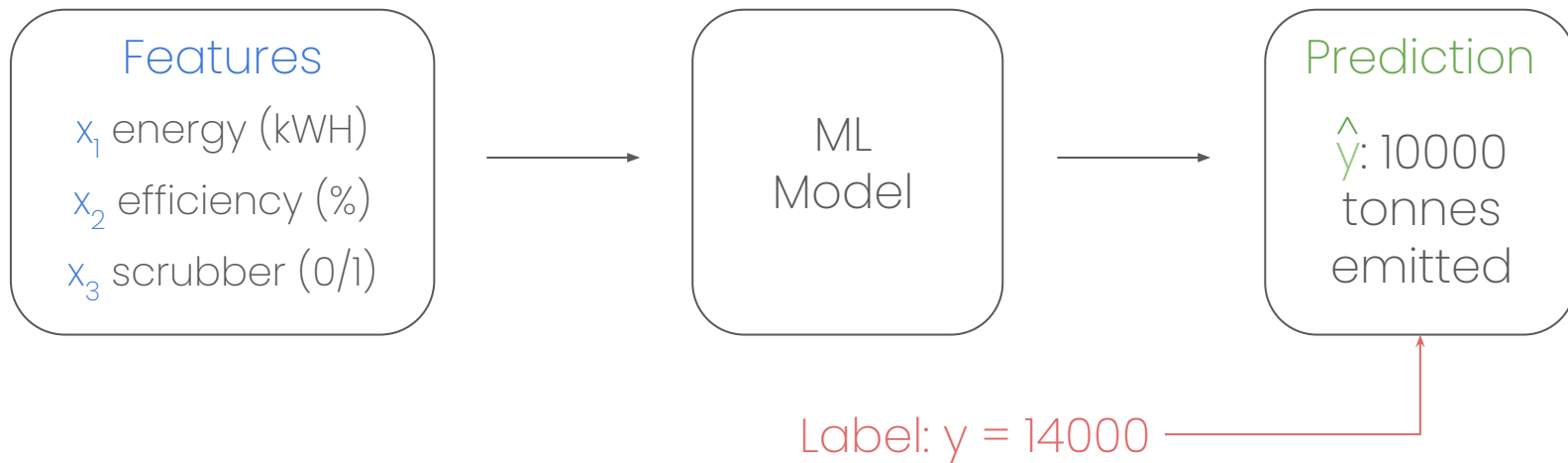


Prediction:
Lynx

Label:
Cat



Ex: Power Plant CO₂ Emissions

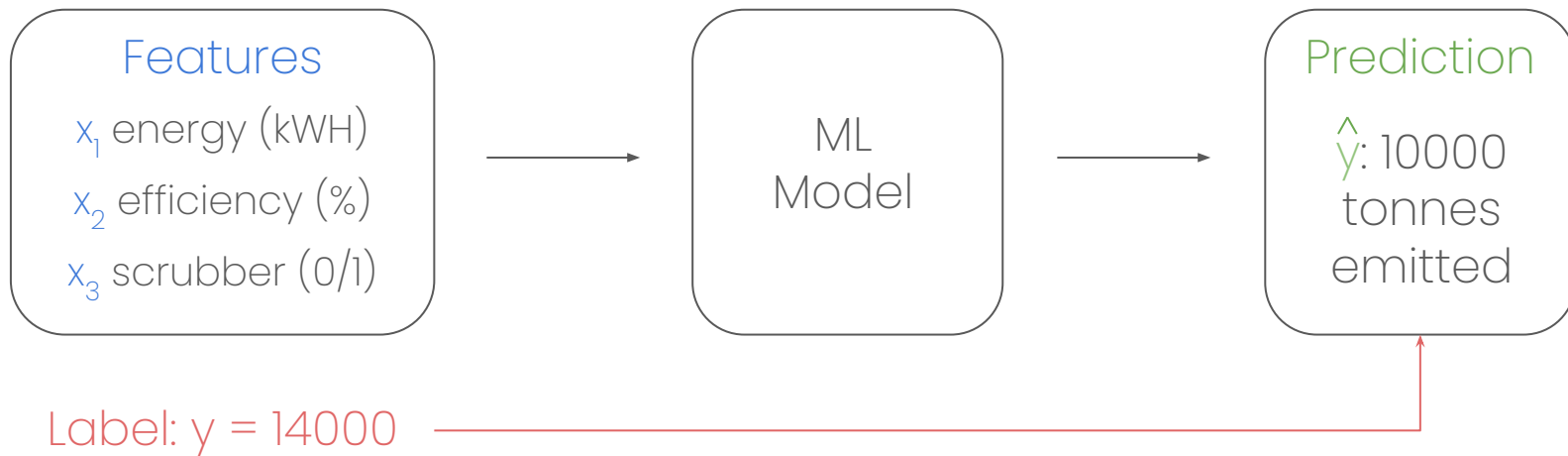


3 Parts of Supervised ML

(Predictions, Error, Optimisation)



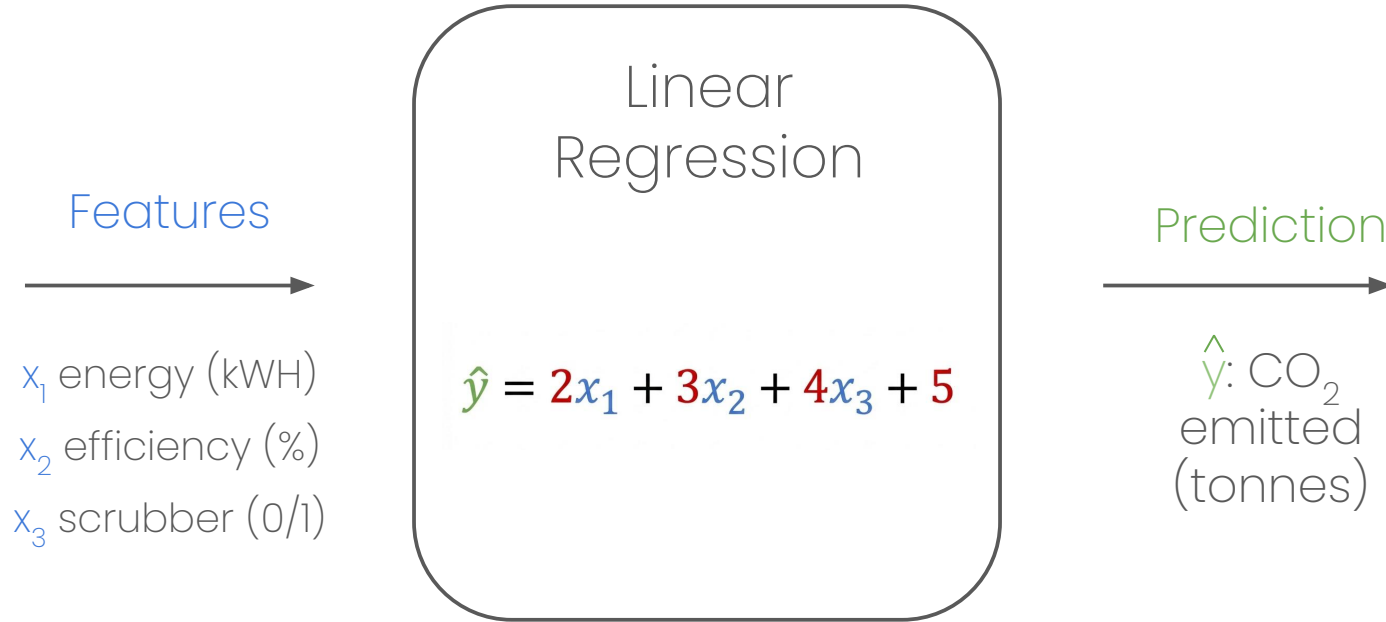
Creating Predictions



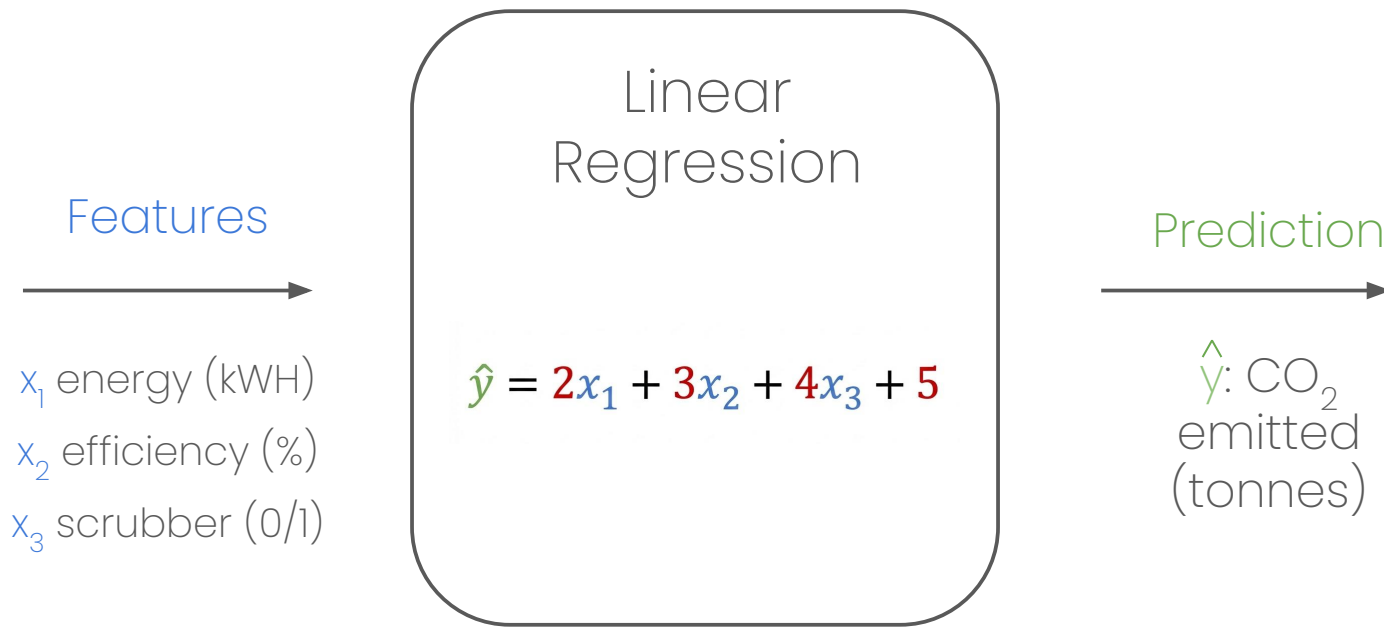
Creating Predictions



Problem solved?



Nope: how to pick **parameters**?

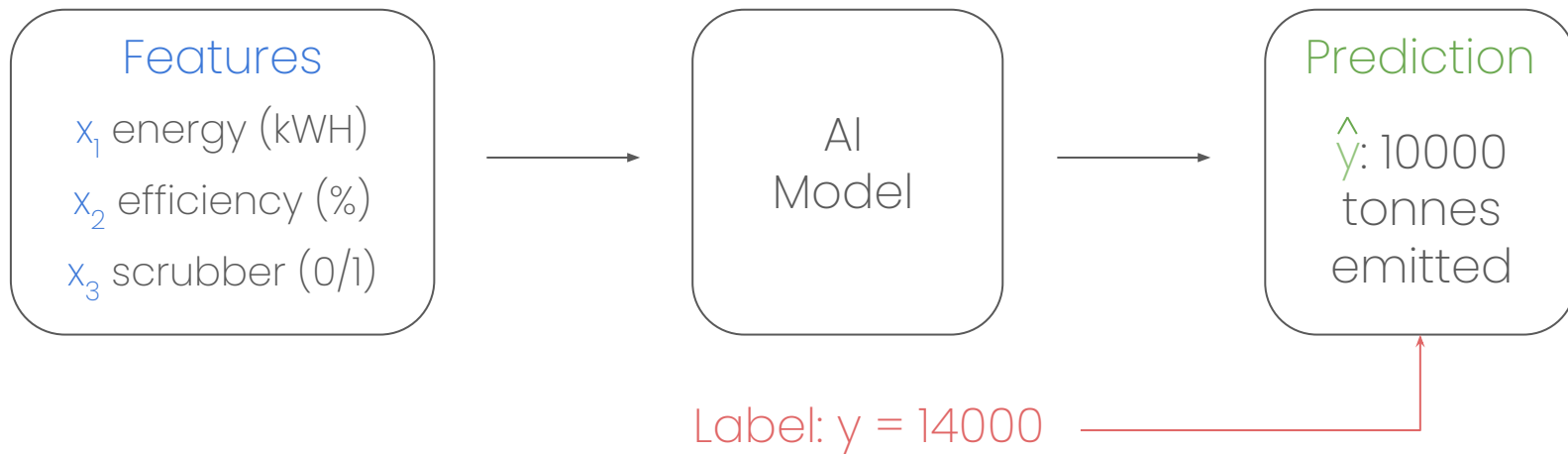


3 Parts of Supervised ML

(Predictions, Error, Optimisation)



How to measure error?



Idea 1: Difference

$$y - \hat{y}$$

$$14000 - 10000$$

$$4000$$



Idea 1: Total Difference

$$(y^{(1)} - \hat{y}^{(1)}) + (y^{(2)} - \hat{y}^{(2)})$$

$$(14000 - 10000) + (8000 - 12000)$$

$$0$$



Idea 2: Total Squared Difference

$$(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2$$

$$(14000 - 10000)^2 + (8000 - 12000)^2$$

32,000,000 🤯



Idea 3: Mean Squared Difference

$$[(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + \dots + (y^{(N)} - \hat{y}^{(N)})^2]/N$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$



3 Parts of Supervised ML

(Predictions, Error, Optimisation)



Consider a single coefficient:

What do we get when we fix everything else constant?

$$\hat{y} = a_1 x_1 + 3x_2 + 4x_3 + 5$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

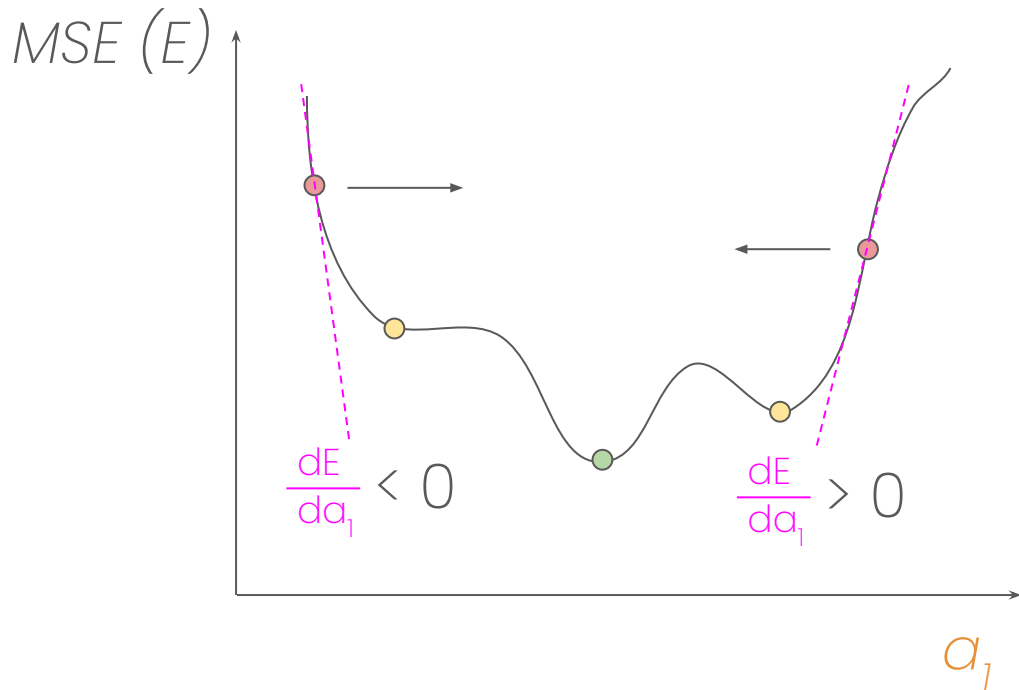


Gradient Descent

$$\hat{y} = a_1 x_1$$

$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

$$a_1 = a_1 - \frac{dE}{da_1}$$

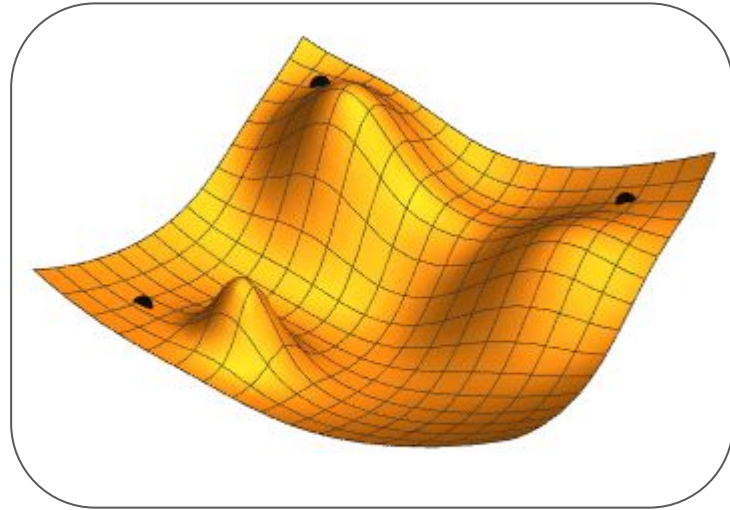


Gradient Descent

$$\hat{y} = a_1 x_1 + a_2 x_2 + \dots$$

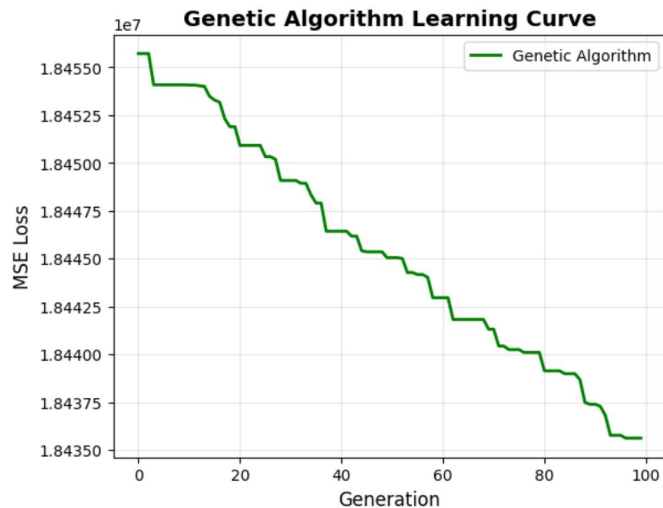
$$MSE = \frac{1}{N} \sum_{k=1}^N (y^{(k)} - \hat{y}^{(k)})^2$$

$$\vec{a} = \vec{a} - \alpha \nabla E(a)$$



Genetic Algorithm (Smart Trial & Error)

1. Make 100 sets of random parameters ($a_1, a_2, a_3, \dots$)
2. Check error for each set
3. Worst 5 sets replaced with new random sets.
4. Other sets get more 'mutations' with higher error. Ex: $\vec{a} = \vec{a} + E \cdot \text{rand}$.



Notebook



End

Next Tuesday (March 10th): Pytorch and
Neural Networks

